CONECTA NEXUS

Guía Estándar de Git, Flujo de Trabajo y Automatización con GitHub Projects

Proyecto e Institución	Conecta Nexus • Universidad Francisco de Paula Santander (UFPS, Cúcuta)
Asignatura y Docente	Desarrollo Basado en Microservicios • Carlos Eduardo Pardo García
Equipo de Desarrollo	José Manuel Pérez (Anorak), Juan Diego Sarmiento, Nelson Enrique Conde
Políticas Clave	Idioma estricto en inglés (commits/branches/PRs) • Sin squash merge • Integración con GitHub Projects

1. Propósito y Estándar Lingüístico

La presente guía técnica establece los lineamientos obligatorios de control de versiones y flujo de trabajo colaborativo para los 9 repositorios del ecosistema Conecta Nexus (frontend, gateway, auth, profile, challenge, application, moderation, notification e infra). El propósito fundamental es garantizar un historial de código autodocumentado, limpio y trazable frente al backlog y la evaluación docente.

Regla de Oro Lingüística: Todos los nombres de ramas (branches), mensajes de confirmación (commits) y títulos/descripciones de Pull Requests (PRs) deben redactarse ESTRICTAMENTE EN INGLÉS. Esta regla aplica de manera uniforme a los tres desarrolladores (Anorak, Juan y Nelson). El uso del inglés en el repositorio sigue las mejores prácticas globales de ingeniería de software, facilita la integración con herramientas de análisis estático, linters automáticos y garantiza una presentación profesional en el portafolio del proyecto.

2. Convención Estricta de Ramas (Branching Strategy)

El repositorio opera bajo un modelo GitFlow adaptado a microservicios. Existen dos ramas principales protegidas de larga duración: 'main' (código estable desplegado en producción) y 'develop' (rama base de integración continua del sprint). Ningún integrante puede hacer push directo sobre 'main' o 'develop'.

Prefijos y Formatos Permitidos: Todas las ramas de trabajo derivan de 'develop' y deben seguir obligatoriamente una de las siguientes tres estructuras estandarizadas:

Tipo de Rama	Formato de Nomenclatura	Descripción y Ejemplo Concreto
Feature (HU)	feature/HU-[ID]-[short-name]	Desarrollo de una Historia de Usuario del backlog. Ej: feature/HU-01-auth-jwt, feature/HU-06-create-challenge
Chore	chore/[short-name]	Tareas de configuración, dependencias, scripts o linters sin lógica de negocio. Ej: chore/setup-podman-compose, chore/configure-ruff-linter
Bugfix	bugfix/issue-[ID]-[short-name]	Corrección de errores funcionales

		detectados durante pruebas de integración. Ej: bugfix/issue-14-token-expiration, bugfix/issue-22-cors-headers
Refactor	refactor/[short-name]	Reestructuración o limpieza de código sin cambiar comportamiento externo. Ej: refactor/auth-service-pydantic-v2, refactor/db-connection-pooling
Hotfix	hotfix/[short-name]	Parche crítico de emergencia generado desde 'main' (solo en incidentes de entrega). Ej: hotfix/fix-production-database-url

Ejemplo de Creación de Rama en Terminal:

```
# 1. Asegurar posición en develop actualizada
git checkout develop
git pull origin develop

# 2. Crear y posicionarse en la nueva rama de feature (en inglés)
git checkout -b feature/HU-01-auth-jwt

# O para una tarea de configuración (chore):
git checkout -b chore/setup-podman-compose
```

3. Convención de Mensajes de Commit (Conventional Commits)

Los commits deben redactarse bajo la convención 'Conventional Commits' en inglés simple, en modo imperativo ('add', 'implement', 'fix', no 'added' o 'implementing') y en minúsculas. El formato estándar es:

```
<type>(<scope>): <short description in imperative mood> [optional issue reference]
```

Tipos válidos de commit (Commit Types):

- **feat:** Nueva funcionalidad para el usuario o sistema (ej. feat(auth): add password hashing with bcrypt).
- **fix:** Corrección de un fallo o bug (ej. fix(challenge): prevent upload of files larger than 10mb).
- **chore:** Mantenimiento, dependencias, tooling o configuración de entorno (ej. chore(repo): add ruff linter configuration).
- **docs:** Documentación en README, Swagger, docstrings o guías (ej. docs(api): update openapi spec for profile endpoints).
- **refactor:** Refactorización de código que no añade features ni corrige bugs (ej. refactor(application): extract matchmaking validation helper).
- **test:** Pruebas unitarias o de integración (ej. test(auth): add unit tests for token expiration).

- **ci:** Configuración de pipelines de GitHub Actions o CI/CD (ej. ci(workflow): add pytest automated quality gate).

Ejemplos Reales de Commits para Conecta Nexus:

```
# Microservicio auth:
git commit -m "feat(auth): implement jwt token generation and validation #12"
git commit -m "test(auth): add pytest test cases for invalid login credentials #12"

# Microservicio profile:
git commit -m "feat(profile): enforce student code immutability rule RB-08 #15"
git commit -m "docs(profile): document habeas data consent law 1581 schema #14"

# Microservicio challenge:
git commit -m "feat(challenge): add file upload endpoint with minio integration #18"
git commit -m "fix(challenge): validate allowed mime types for pdf attachments #18"

# Configuración e infraestructura:
git commit -m "chore(infra): add docker-compose definition for local postgres
databases"
git commit -m "ci(actions): enforce branch protection status checks on develop"
```

4. Política de Integración: Prohibición de Squash Merge

Política Oficial del Equipo: Para el alcance y evaluación académica de Conecta Nexus, QUEDA ESTRICTAMENTE PROHIBIDO EL USO DE SQUASH MERGE. En la configuración de los Pull Requests se debe utilizar exclusivamente la opción 'Create a merge commit' (o 'Rebase and merge' si no hay bifurcación, pero el estándar del equipo es 'Create a merge commit' con '--no-ff').

Justificación Técnica y Metodológica:

- **1. Trazabilidad Histórica y Autodocumentación:** En una auditoría de código docente y en la defensa del proyecto, es fundamental que el árbol de commits en 'develop' y 'main' muestre la evolución paso a paso, los micro-avances atómicos y la co-responsabilidad entre los Code Owners. El Squash Merge aplasta todo en un solo commit genérico, destruyendo los mensajes atómicos y ocultando el esfuerzo diario.
- **2. Atribución Clara de Contribuciones:** Cada microservicio tiene dos personas asignadas. Al mantener los commits individuales, los gráficos de autoría de GitHub y los comandos 'git log' y 'git blame' reflejan con precisión los aportes de Anorak, Juan y Nelson sin que el autor del merge opaque al autor del código.
- **3. Reversión Granular de Cambios:** Si un commit introduce una regresión, un historial preservado permite aislar el cambio exacto mediante 'git bisect' o revertir un commit puntual con 'git revert' sin tener que deshacer toda la Historia de Usuario.

5. Flujo de Integración Nativa con GitHub Projects

El tablero 'Conecta Nexus - Core Management' a nivel de organización se actualiza de manera natural a través del flujo de Git, sin necesidad de que los desarrolladores muevan tarjetas de forma manual repetitiva.

Fase del Flujo	Acción en Git / GitHub	Palabra Clave / Comando	Estado en Project
1. Planificación	Se crea el Issue formal en el repo hijo vinculado al	Issue #[ID] creado con Story	**To Do**

	Project.	Points y Sprint asignado.	
2. Inicio Desarrollo	El desarrollador crea la rama 'feature/HU-[ID]-...' y se autoasigna.	git checkout -b feature/HU-01-auth-jwt (Mover tarjeta a 'In Progress').	**In Progress**
3. Revisión de Código	Se abre el Pull Request hacia 'develop' referenciando el issue.	En PR description: Closes conecta-nexus/auth#[ID]	**In Review**
4. Aprobación y Merge	El Code Owner aprueba el PR. Se hace 'Merge commit' (SIN squash).	Merge pull request -> GitHub detecta 'Closes #[ID]'.	**Done (Automático)**

Ejemplo Completo de Ciclo de Vida de una Tarea:

Caso Práctico: Supongamos que Anorak y Nelson están trabajando en el repositorio 'auth' para la historia HU-01 (Issue #5 en 'conecta-nexus/auth'):

```
# PASO 1: Desde develop actualizado, crear rama de feature
git checkout develop
git pull origin develop
git checkout -b feature/HU-01-auth-registration

# PASO 2: En el tablero de GitHub Projects, mover la tarjeta a 'In Progress'

# PASO 3: Realizar commits atómicos en inglés con conventional commits
git add src/models/user.py
git commit -m "feat(auth): create user database model with role enum #5"

git add src/services/auth.py
git commit -m "feat(auth): add password hashing and registration service #5"

git add tests/test_registration.py
git commit -m "test(auth): add pytest cases for duplicate email registration #5"

# PASO 4: Subir la rama a GitHub
git push -u origin feature/HU-01-auth-registration
```

6. Estándar de Pull Requests y Plantilla de Revisión

Al abrir el Pull Request en GitHub, el título y la descripción deben redactarse en inglés. La base debe ser siempre 'develop' (o 'main' al final del sprint para el release).

Formato del Título de PR:

```
<type>(<scope>): <concise description in English> (HU-[ID])

# Ejemplos:
feat(auth): implement user registration and role assignment (HU-01)
feat(challenge): add attachment streaming endpoint (HU-07)
chore(infra): add postgres local initialization script
```

Plantilla Obligatoria de Descripción de PR:

```
## Summary
Implemented the multi-role user registration endpoint for Students and Companies,
including password hashing with bcrypt and input validation using Pydantic.

## Related Issue
Closes conecta-nexus/auth#5

## Type of Change
- [x] feat: New feature
- [ ] fix: Bug fix
- [ ] chore: Tooling or maintenance

## Checklist
- [x] Code follows the team naming standards and is written in English.
- [x] Unit tests were added and coverage is >= 80%.
- [x] All status checks in GitHub Actions passed (green).
- [x] Local Podman/Docker containers run without errors.
- [x] Merge method set to 'Create a merge commit' (NO SQUASH).

## Reviewer (Code Owner)
@NelsonConde (or peer assigned according to the Code Owners matrix)
```

7. Reglas de Fusión (Merge Rules) y Cheat Sheet del Equipo

Candados Obligatorios de Merge: Antes de presionar el botón de fusión, el autor y el revisor deben verificar los siguientes tres candados:

- **1. Aprobación Paritaria:** Al menos un compañero asignado como Code Owner debe haber revisado el código y seleccionado 'Approve'.
- **2. Status Checks en Verde:** Todos los checks de GitHub Actions (tests unitarios, cobertura >= 80%, linter) deben estar en verde.
- **3. Selección Estricta de 'Merge Commit':** En el menú desplegable del botón de merge de GitHub, seleccionar SIEMPRE 'Create a merge commit'. Nunca seleccionar 'Squash and merge'.

Resumen Rápido de Comandos para el Día a Día (Cheat Sheet):

```
# Iniciar trabajo en nueva tarea:
git checkout develop && git pull origin develop
git checkout -b feature/HU-XX-short-name

# Confirmar cambios atómicos:
git add <archivos>
git commit -m "feat(<scope>): description in English #<issue-id>"

# Subir rama y crear PR:
git push -u origin feature/HU-XX-short-name

# Actualizar rama local con cambios que otros subieron a develop:
git checkout develop && git pull origin develop
git checkout feature/HU-XX-short-name
git merge develop
```

```
# Revertir un commit defectuoso manteniendo historial (sin reescribir):
git revert <commit-sha>
```